Que No Truene el Día de la Entrega

P. L. Alvarez Gonzalez
7690 - 22 - 5839
Universidad Mariano Gálvez
Seminario de Tecnologías de Información
palvarezg1@miumg.edu.gt

Resumen

El objetivo de este trabajo es explicar de forma sencilla cómo se garantiza la calidad en un proyecto de tecnologías de información. Muchas veces se piensa que la calidad es algo que se revisa al final, cuando el sistema ya está terminado, pero la investigación muestra que es un trabajo constante que empieza desde el primer día. Para lograrlo se consultó bibliografía en ingeniería de software y en gestión de proyectos, y se ordenó la información siguiendo el camino natural de un proyecto, desde los requisitos hasta la entrega y el mantenimiento. Como resultado se hizo una explicación clara de las prácticas que más ayudan a que un proyecto salga bien, como definir requisitos claros, probar desde temprano, revisar el código entre varias personas, medir la calidad con estándares conocidos y aprender de los errores. Se concluye que la calidad no depende de una sola herramienta, sino de que todo el equipo se comprometa con hacer las cosas bien desde el inicio.

Palabras clave: calidad de software, gestión de proyectos, pruebas, requisitos, mejora continua.

Desarrollo del tema

Antes de hablar de cómo garantizar la calidad, conviene tener claro qué significa. En un proyecto de TI, calidad no quiere decir que el sistema sea perfecto ni que tenga todas las funciones imaginables. Quiere decir que el producto cumple lo que el cliente pidió, que funciona sin fallar cuando se usa y que se puede mantener sin dolores de cabeza en el futuro. Sommerville, en su libro del 2016, explica que la calidad del software tiene que ver tanto con lo que el sistema hace como con qué tan bien lo hace, por ejemplo qué tan rápido responde, qué tan seguro es y qué tan fácil resulta de usar.

El primer paso para tener calidad es entender bien lo que se tiene que construir. Un montón de proyectos fracasan no porque el código esté mal escrito, sino porque se construyó algo distinto a lo que el cliente necesitaba. Por eso la recopilación de requisitos es tan importante. Los requisitos deben quedar por escrito, ser claros y estar aprobados por el cliente antes de empezar a programar. Si algo no se entiende, es mejor preguntar en ese momento que descubrir el malentendido cuando ya se gastó la mitad del presupuesto.

El segundo paso es escoger una metodología de trabajo que se adapte al proyecto. Ya sea una metodología ágil como Scrum o una más tradicional como cascada, lo importante es que el equipo tenga reglas claras sobre cómo se trabaja, cómo se revisan los avances y cómo se manejan los cambios. En las metodologías ágiles, por ejemplo, el cliente ve avances cada pocas semanas, y eso permite detectar errores de rumbo a tiempo y no al final.

El tercer paso es planificar la calidad desde el inicio. El Project Management, en su guía, plantea que la gestión de la calidad incluye planificar qué estándares se van a cumplir, asegurar que el trabajo realmente los cumpla y controlar los resultados con mediciones. En otras palabras, el

equipo define desde el principio qué significa que algo está bien hecho y después revisa si de verdad se está logrando. Para esto ayuda mucho apoyarse en estándares reconocidos, como la norma ISO/IEC 25010, que describe las características que debe tener un producto de software de calidad, entre ellas la funcionalidad, el rendimiento, la seguridad, la facilidad de uso y la facilidad de mantenimiento.

El cuarto paso son las pruebas, que son probablemente lo que más se asocia con la calidad. Probar no es solo darle clic a la aplicación para ver si truena. Existen pruebas unitarias, que revisan que cada pedacito de código haga lo que debe, pruebas de integración, que revisan que las partes funcionen bien juntas, y pruebas de aceptación, donde el cliente confirma que el sistema hace lo que pidió. Pressman y Maxim, en su libro, insisten en que entre más temprano se encuentra un error, más barato sale corregirlo. Un fallo detectado mientras se escribe el código se arregla en minutos, pero el mismo fallo encontrado cuando el sistema ya está en producción puede costar muchísimo más, tanto en dinero como en la confianza del cliente.

El quinto paso es revisar el trabajo entre compañeros. Las revisiones de código consisten en que otra persona del equipo lea lo que se programó antes de aceptarlo. Esto ayuda a encontrar errores que el autor ya no ve porque está muy metido en su propio trabajo, y además sirve para que todo el equipo aprenda y mantenga un estilo parecido. Junto con esto, hoy en día es muy común automatizar las pruebas y la integración del código, de forma que cada vez que alguien sube un cambio se corren las pruebas solas y se avisa de inmediato si algo se rompió.

El sexto paso es controlar los defectos y la configuración del proyecto. Cuando se encuentra un error, no basta con arreglarlo y seguir. Hay que registrarlo, entender de dónde vino y ver si se puede evitar que vuelva a pasar. Del mismo modo, hay que tener control de las versiones del código y de los documentos, para que todos trabajen sobre lo mismo y no se pierdan cambios importantes.

Por último, la calidad no termina cuando se entrega el sistema. Después de la entrega viene el mantenimiento y ahí es donde se nota si el trabajo se hizo bien. Por eso es importante cerrar cada proyecto o cada etapa revisando qué salió bien, qué salió mal y qué se puede mejorar para la próxima vez. Esa práctica de mejora continua es lo que hace que un equipo cada vez cometa menos errores.

Observaciones y comentarios

Durante esta investigación se pudo notar que la mayoría de problemas de calidad vienen de la necesidad que todo se haga rapido y de la falta de comunicación, más que de falta de conocimiento técnico. Muchos equipos saben que deberían probar y revisar, pero se saltan esos pasos cuando se acerca la fecha de entrega, y después pagan el doble por eso. También se observó que ninguna herramienta ni estándar sustituye a un equipo que se comunica bien con el cliente y entre sí. Una limitación de este trabajo es que se enfocó en las prácticas generales y no incluyó un caso real de un proyecto con medición de resultados, lo cual podría ser un buen tema para ampliar en el futuro.

Conclusiones

1. La calidad de un proyecto de TI se construye desde el primer día y no se revisa solo al final.
2. Tener requisitos claros y aprobados por el cliente evita construir algo que nadie pidió.
3. Probar temprano y de varias formas reduce mucho el costo de corregir errores.

4. Los estándares, las revisiones entre compañeros y la mejora continua ayudan a que el equipo mantenga la calidad en el tiempo.

Referencias

International Organization for Standardization. (2011). *ISO/IEC 25010:2011 Systems and software engineering: Systems and software quality requirements and evaluation: System and software quality models*.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.

Project Management Institute. (2021). *Guía de los fundamentos para la dirección de proyectos: Guía del PMBOK* (7.ª ed.). Project Management Institute.

Sommerville, I. (2016). *Ingeniería de software* (10.ª ed.). Pearson Educación.

Repositorio: https://github.com/PLAlvarez11/seminario-encuesta